

Fixing Galaga Bézier Curves

<https://gemini.google.com/app/6c8d0e682087458a>

User prompt: I have a Pygame Galaga clone. Enemies enter and dive along cubic Bézier curves drawn by a `bezier(p0, p1, p2, p3, t)` function. The curve bulges/overshoots mid-flight even though start and end points are correct. Compare my `bezier()` to the standard cubic Bézier formula and show me the corrected version of the function with the wrong term fixed. Explain briefly what the correct curve should look like compared to the buggy one.

Response: Here is the comparison between the standard cubic Bézier formula and the buggy version, along with the corrected function and an explanation of the visual fix.

1. Comparison with the Standard Cubic Bézier Formula

The mathematical definition for a standard cubic Bézier curve using start point P_0 , control points P_1 and P_2 , and end point P_3 is:

$$B(t) = (1 - t)^3 P_0 + 3(1 - t)^2 t P_1 + 3(1 - t) t^2 P_2 + t^3 P_3$$

Let $u = 1 - t$. The four weighted terms are:

1. **Start term:** $u^3 P_0$ (ensures $B(0) = P_0$)
2. **First control term:** $3u^2 t P_1$ (u is squared, t is linear)
3. **Second control term:** $3u t^2 P_2$ (u is linear, t is squared)
4. **End term:** $t^3 P_3$ (ensures $B(1) = P_3$)

The Bug

In the buggy implementation, an incorrect exponent or duplicated term in the middle coefficients (such as using $3 * u * t * t$ for both middle terms or swapping the powers) breaks the **partition of unity** (where the sum of all coefficients equals 1 for any t). Because the blending weights do not sum correctly across the timeline, the curve loses its smooth interpolation, causing it to excessively warp, bulge, or overshoot mid-flight while still correctly landing on P_0 and P_3 .

2. Corrected `bezier()` Function

To fix the bug, ensure that the first control point (P_1) uses $u^2 t$ and the second control point (P_2) uses $u t^2$:

```
def bezier(p0, p1, p2, p3, t):
    u = 1 - t
    x = u ** 3 * p0[0] + 3 * u * u * t * p1[0] + 3 * u * t * t * p2[0] + t ** 3 * p3[0]
    y = u ** 3 * p0[1] + 3 * u * u * t * p1[1] + 3 * u * t * t * p2[1] + t ** 3 * p3[1]
    return pygame.Vector2(x, y)
```

3. What the Curves Look Like

- **Buggy Curve:** Bulges aggressively or whips sideways mid-flight with erratic acceleration because the weighting assigned to the control points is unbalanced. Enemies might swing wildly away from their intended entry arcs.
- **Correct Curve:** Sweeps smoothly and predictably along a natural arc, transitioning gracefully from the entry point through the control tangents toward the final formation or dive destination.

User prompt: For the same game, implement `enemy_tint(kind)`. It's called in `draw()` as: `color = enemy_tint(enemy.kind)` or `ENEMY_COLORS[enemy.kind]`. `kind` is "boss", "red", or "blue". I want enemies recolored by wave number so later waves look visually distinct. The current wave isn't passed into `enemy_tint`, but `spawn_wave(wave)` builds the enemies. Show me the code needed — the module-level variable, the change to `spawn_wave`, and the `enemy_tint` function — so wave 1 keeps default colors and wave 2+ get gradually brighter, clamped to 0-255. For the same game, implement `on_wave_start(wave)`. It's called from `spawn_wave(wave)`. I want a "WAVE N" banner to appear for about 1.5 seconds at the start of every wave. Show me the code for: (1) the module-level state, (2) `on_wave_start`, (3) the addition to `Game.update` that decrements the timer each frame, and (4) the addition to `Game.draw` that renders the banner centered above the middle of the screen.

Response: Here is the code needed to implement both `enemy_tint(kind)` and `on_wave_start(wave)` for your Galaga clone, based on the structure already present in your files.

Part 1: Enemy Tenting (`enemy_tint` & `spawn_wave`)

Your provided code actually already has the template for `CURRENT_WAVE`, `enemy_tint`, and `on_wave_start` stubbed out. You can complete them as follows:

1. Module-Level Variable & Function (`game.py`)

```
CURRENT_WAVE = 1
```

```
def enemy_tint(kind):
    """Return an (r, g, b) colour override for an enemy kind, or None for the default."""
    if CURRENT_WAVE < 2:
        return None
    base = ENEMY_COLORS[kind]
    shift = min((CURRENT_WAVE - 1) * 15, 90)
    return tuple(min(255, c + shift) for c in base)
```

2. Change to `spawn_wave(wave)`

Update `spawn_wave` to update the global `CURRENT_WAVE` variable:

```
def spawn_wave(wave):
    global CURRENT_WAVE
    CURRENT_WAVE = wave
    enemies, index = [], 0
    layout = [("boss", 4, 90), ("red", 8, 130), ("red", 8, 170), ("blue", 8, 210)]
    for kind, count, y in layout:
        for i in range(count):
            x = WIDTH / 2 + (i - (count - 1) / 2) * 52
            enemies.append(Enemy(kind, (x, y), index, index % 2 == 0))
            index += 1
    on_wave_start(wave)
    return enemies
```

Part 2: Wave Start Banner (`on_wave_start`, Update, and Draw)

1. Module-Level State

```
WAVE_BANNER = None # Stores a tuple of (text, remaining_seconds)
```

2. `on_wave_start(wave)` Function

```
def on_wave_start(wave):
    """Called at the start of every wave; sets up the wave announcement banner."""
    global WAVE_BANNER
    WAVE_BANNER = (f"WAVE {wave}", 1.5)
```

3. Addition to `Game.update`

Add this snippet inside `Game.update()` (right after `self.time += dt`):

```
global WAVE_BANNER
if WAVE_BANNER is not None:
    text, remaining = WAVE_BANNER
    remaining -= dt
    WAVE_BANNER = (text, remaining) if remaining > 0 else None
```

4. Addition to `Game.draw`

Add this snippet inside `Game.draw()` (right where UI elements or text like HUD/Game Over are rendered):

```
if WAVE_BANNER is not None:
    text, _ = WAVE_BANNER
    banner = font.render(text, True, (255, 255, 120))
    screen.blit(banner, banner.get_rect(center=(WIDTH // 2, HEIGHT // 2 - 60)))
```

User prompt: For the same game, implement `shield_charges(wave)`. It's called as: `self.shield = shield_charges(self.wave)` or `0`. The `hit_player()` method already consumes `self.shield`. I want 1 shield charge granted every 3rd wave (waves 3, 6, 9...). Show me the code for `shield_charges`. Also tell me what visibly changes on screen when the shield absorbs a hit, so I can demonstrate it in a video.

Response: 1. Code for shield_charges(wave)

The function to grant 1 shield charge every 3rd wave (waves 3, 6, 9, etc.) is implemented as follows:

```
def shield_charges(wave):  
    """Return how many hits the player's shield can absorb this wave, or None to disable the shield."""  
    return 1 if wave % 3 == 0 else 0
```

2. What Visibly Changes on Screen When the Shield Absorbs a Hit

When playing on a shield wave (such as Wave 3 or Wave 6) and the player's ship is struck by an enemy bullet or a diving enemy, the following visual and game-state changes occur:

- **No Loss of Lives or Ships:** Your life count and active ship count remain completely unchanged, preventing a game-over or downgrade penalty for that hit.
 - **Ship Flickering (Invulnerability):** The ship immediately starts blinking/flickering on screen. This visual flashing is triggered by the `self.invulnerable = 1.0` setting inside `hit_player()`, signaling that temporary protection has absorbed the blow.
 - **Shield Depletion:** The single shield charge for that wave is consumed, so subsequent hits during that same wave will deal normal damage (unless you advance to the next shield wave).
-